

Decentralized Authorized Artificial Intelligence Queries in Cloud-based Internet of Things

Mingyang Zhao, Junhan Qin, Xixi Zheng, Wenbin Zhai, Yiming Cao, and Bin Xiao, *Fellow, IEEE*

Abstract—Artificial intelligence (AI) queries leverage AI models to generate predictive results and provide deeper insights than traditional match-based queries, which have shown significant potential for Internet of Things (IoT) applications. For example, in smart healthcare applications, users utilize IoT devices (e.g., watches or phones) to collect their health data, and doctors execute AI queries for disease prediction. Currently, to comply with the right to restriction of processing under laws and regulations, existing AI query works have to trust and depend on centralized cloud servers to enforce users' access control policies for authorized AI queries. However, this is infeasible for decentralized IoT environments, since these cloud servers, as enforcers, are not trusted and might bypass users' policies to break data security.

In this paper, we introduce Decart, a decentralized authorized AI query scheme for cloud-based IoT applications. Different from existing works, while utilizing cloud servers for data outsourcing and AI queries, Decart enables users to formulate account-level policies that regulate which accounts can execute AI queries, and leverages cryptographic tools to enforce these policies on all entities, including these cloud servers. Meanwhile, Decart supports multiple AI models, such as decision trees and neural networks, and revocation functionalities to revoke malicious users. Further, we optimize the size of users' stored parameters of Decart ($\mathcal{O}(n^2)$) to introduce a more efficient variant as Decart* ($\mathcal{O}(n)$), where n is related to the number of users. Security analysis demonstrates the security in decentralized settings. Experiments show that when running AI models with millions of parameters, compared to naively assembling existing techniques for authorized AI queries, Decart and Decart* achieve $10\times$ and $20\times$ reduction in computation costs and $10\times$ and $20\times$ reduction in communication costs. The source code is outsourced at Anonymous GitHub¹.

Index Terms—Internet of Things, Cloud Servers, Artificial Intelligence Queries, Authorization, and Decentralization.

I. INTRODUCTION

xxx

xxx

xxx

xxx

-
-
-

xxxx

M. Zhao, J. Qin, X. Zheng, W. Zhai, X. Cai, and B. Xiao are with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong. Email: {25019897r}@connect.polyu.hk and {b.xiao}@polyu.edu.hk.

Corresponding author: Bin Xiao (Email: b.xiao@polyu.edu.hk).

This work was supported in part by the National Natural Science Foundation of China (NSFC) / Research Grants Council (RGC) Collaborative Research Scheme (Grant No. 62461160332 & CRS_HKUST602/24).

¹xxx

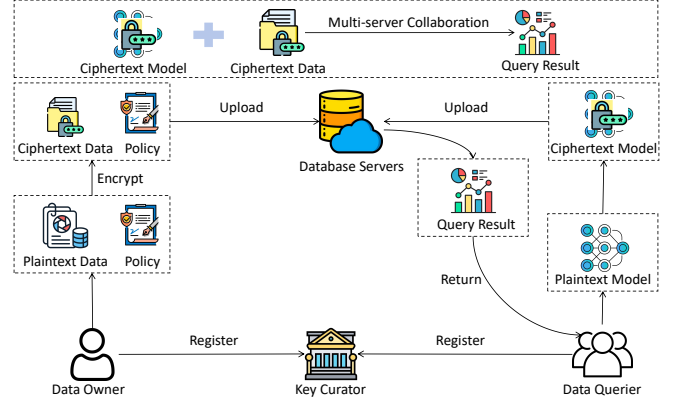


Fig. 1: The system model in this paper.

II. RELATED WORKS

III. PRELIMINARIES

A. Artificial Intelligence and Prediction Queries

B. Bilinear Maps and Hardness Assumptions

C. Registration-based Encryption

D. Threshold Homomorphic Encryption

IV. PROBLEM FORMULATION

In this section, we present the system model and threat model in this paper. Then, we summarize the design goals.

A. System Model

As shown in Fig. 1, the system model in Web 3.0 artificial intelligence query scenarios consists of the key curator, data owners, cloud servers, and data queriers.

- **Key Curator.** Without any secrets, the key curator is a public platform for collecting user keys and maintaining system parameters.
- **Data Owner.** Data owners, as Web 3.0 users, store data records to the cloud server and formulate query policies for managing AI queries.
- **cloud server.** As cloud servers, the cloud server is responsible for storing the data owners' data records and executing data queriers' query services.
- **Data Querier.** As Web 3.0 users, data queriers use the cloud server to execute AI queries.

B. Threat Model

The semi-trusted assumption is adopted in this paper, meaning that all entities in the system model are semi-trusted. For instance, the key curator is curious about all users' secret keys, the data owners' data records, and data queriers' AI models. Data owners are curious about other data owners' data records and data queriers' AI models. Data queries attempt to break the query policy for unauthorized AI queries. The cloud server attempts to break the query policy to smuggle AI queries to unauthorized data queries, and is curious about the data queriers' AI models. Collusion attacks might exist between these entities, and solutions against collusion attacks are discussed in Section VI.

C. Design Goals

Without leaking data owners' data records and data queriers' AI models, the cloud server combines data owners' query policies to execute AI queries and returns the query results to data queriers. Specifically, we summarize the design goals in terms of functionality, security, and efficiency.

- **Functionality.** Data owners can control which data queriers can execute AI queries.
- **Security.** Without any centralized entity, data owners' query policies are enforceable to all entities, including the cloud server. This means, only data queriers whose identities satisfy the data owners' query policies can execute AI queries, which prevents the cloud server from smuggling AI queries to unauthorized data queriers. When malicious users are reported, the key curator can execute revocation functionalities to remove them.
- **Efficiency.** Operations with heavy computations, such as the AI query processes, occur on the cloud server, instead of data owners and data queriers. Meanwhile, towards multiple different data queriers, data owners simply need to generate a $|m|$ -sized ciphertext, rather than generating a $|m|$ -sized ciphertext for each data querier, where $|m|$ denotes the size of data records.

V. OUR PROPOSED Decart AND Decart*

Combining the above idea and definition of our schemes, we present the detailed constructions below.

A. Main Idea for Technical Challenges

Idea for Transferring the Focus of Registration-based Encryption from Data to AI Query Control.

Idea for Reducing Communication Costs towards Multi-user Scenarios.

Idea for Reducing the Size of Owners' Stored Parameters

B. Detailed Construction for Decart

Setup $(\lambda) \rightarrow (crs, pp, aux)$: Given a security parameter λ , the system start the trusted initialization. First, the system generates a bilinear group Ψ as (p, g, G, G_T, Z_p, e) , where $e(G, G) \rightarrow G_T$. Let the parameter $N \in Z_p$ as the maximum

number of users, $n \in Z_p$ as the number of users in a block, and $B = \lceil \frac{N}{n} \rceil$ as the number of blocks. Under a trusted setup, the system samples n random values $\{z_i \in Z_p\}_{i=1}^n$, and computes the following parameters.

$$h_i = g^{z_i}, \quad i = 1, 2, \dots, n,$$

$$H_{i,j} = g^{z_i \cdot z_j}, \quad i, j = 1, 2, \dots, n, \text{ and } i \neq j.$$

Finally, the system selects a hash function $H[G_T] \rightarrow \{0, 1\}^*$ and initializes the common reference string crs , the public parameter pp , and the auxiliary parameter aux as follows.

$$crs = (\Psi, N, B, n, \{h_i\}_{i \in [n]}, \{H_{i,j}\}_{i,j \in [n] \& i \neq j}, H[\cdot]),$$

$$pp = (C_{(1)} = 1, C_{(2)} = 1, \dots, C_{(B)} = 1) \in G,$$

$$aux = (L_1 = \{1\}, L_2 = \{1\}, \dots, L_N = \{1\}) \in G.$$

KeyGen $(u_{id}) \rightarrow (sk_{id}, pk_{id}, pap_{id})$: Given the user identity $u_{id} \in Z_p$, the user samples a random value $x_{id} \in Z_p$, and sets $u'_{id} = (u_{id} \bmod n) + 1$. Then, the user generates his/her own secret key sk_{id} , public key pk_{id} , and personal auxiliary parameter pap_{id} as follows. Next, the user sends the tuple $(u_{id}, pk_{id}, pap_{id})$ to the key curator for registration.

$$sk_{id} = x_{id}, \quad pk_{id} = h_{u'_{id}}^{x_{id}},$$

$$pap_{id} = (H_{1,u'_{id}}^{x_{id}}, \dots, H_{u'_{id}-1,u'_{id}}^{x_{id}}, \phi, H_{u'_{id}+1,u'_{id}}^{x_{id}}, \dots, H_{n,u'_{id}}^{x_{id}}).$$

Register $(u_{id}, pk_{id}, pap_{id}) \rightarrow (pp', aux')$: Receiving tuple $(u_{id}, pk_{id}, pap_{id})$, the key curator checks the user identity u_{id} and personal auxiliary parameter pap_{id} as follows.

$$e(pk_{id}, h_1) \stackrel{?}{=} e(pap_{id,1}, g), \dots, e(pk_{id}, h_n) \stackrel{?}{=} e(pap_{id,n}, g),$$

where $u'_{id} = (u_{id} \bmod n) + 1$. If the above checking process holds, the key curator computes the parameter $k = \lceil \frac{u_{id}+1}{n} \rceil$ as the block number belonging to u_{id} . Then, the key curator updates $C'_{(k)}$ as $C_{(k)} \cdot pk_{id}$. Apart from $C_{(k)}$, other parameters in public parameter pp remain the same, i.e., the updated pp' is $(C_{(1)}, C_{(2)}, \dots, C'_{(k)}, \dots, C_{(B)})$. Next, for $j \in \{(k-1)n+1, (k-1)n+2, \dots, (k-1)n+n\} \setminus (u_{id}+1)$, the key curator parses L_j as $(O_{j,1}, O_{j,2}, \dots, O_{j,o})$ and updates L_j as $L_j \cup \{O_{j,o+1} = O_{j,o} \cdot pap_{id}\}$, where o denote the current size of L_j and $i = j - (k-1)n$. Apart from $j \in \{(k-1)n+1, (k-1)n+2, \dots, (k-1)n+n\} \setminus (u_{id}+1)$, other parameters in the auxiliary parameter aux remains the same.

Encrypt $(\mathcal{P}, \{m_i\}_{i=1}^{n_m}) \rightarrow C_m$: The user formulates the access policy $\mathcal{P}$ as a series of user identities $\{u_{id,i}\}_{i=1}^{n_p}$, where n_p denotes the size of the access policy. Then, the user samples a random value $\alpha \in Z_p$ and a homomorphic encryption key tuple (pk_h, sk_h) with the homomorphic encryption as $Enc[\cdot]$, homomorphic decryption as $Dec[\cdot]$, and homomorphic evaluation as $Eval[\cdot]$. Next, the user samples two random value $(\beta, \gamma) \in Z_p$, splits the homomorphic secret key sk_h as two shares $(sk_{h,s}, sk_{h,u})$, and encrypt the data m_i in the

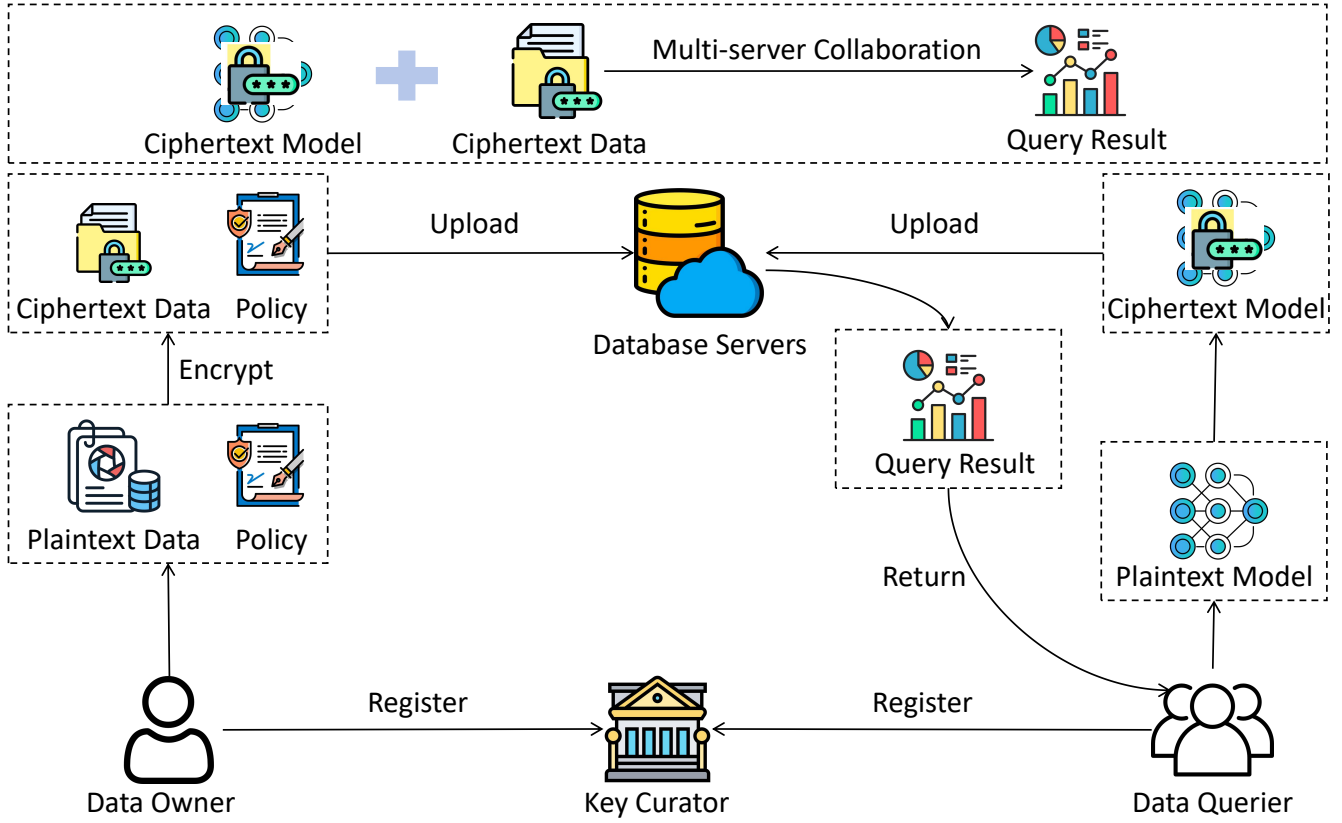


Fig. 2: The workflow of Decart.

dataset $\{m_i \in \{0, 1\}^* \}_{i=1}^{n_m}$ as follows, where n_m denotes the number of data records.

$$\begin{aligned}
 k_i &= \lceil \frac{u_{id,i} + 1}{n} \rceil, \quad i = 1, 2, \dots, n_p, \\
 c_{1,i} &= C_{(k_i)}, \quad i = 1, 2, \dots, n_p, \\
 u'_{id,i} &= (u_{id,i} \bmod n) + 1, \quad i = 1, 2, \dots, n_p, \\
 c_{2,i} &= e(C_{(k_i)}, h_{u'_{id,i}})^\gamma, \quad i = 1, 2, \dots, n_p, \\
 c_3 &= g^\gamma, \\
 c_{4,i} &= H[e(h_{u'_{id,i}}, h_{u'_{id,i}})^\gamma] \oplus \beta, \quad i = 1, 2, \dots, n_p, \\
 c_5 &= H[e(g, g)^\beta] \oplus (pk_h || sk_{h,u}), \\
 c_{6,i} &= Enc[pk_h, m_i], \quad i = 1, 2, \dots, n_m.
 \end{aligned}$$

Finally, the user packs the ciphertext C_m as $(\mathcal{P}, \{c_{1,i}\}_{i=1}^{n_p}, \{c_{2,i}\}_{i=1}^{n_p}, c_3, \{c_{4,i}\}_{i=1}^{n_p}, \{c_{5,i}\}_{i=1}^{n_m}, c_5, \{c_{6,i}\}_{i=1}^{n_m})$, and sends the tuple $(C_m, sk_{h,s})$ to the cloud server. Then, the cloud server retains the policy-related parameters $(\mathcal{P}, \{c_{1,i}\}_{i=1}^{n_p}, \{c_{2,i}\}_{i=1}^{n_p}, c_3, \{c_{4,i}\}_{i=1}^{n_p}, c_5)$ locally and uploads the data-related parameters $\{c_{6,i}\}_{i=1}^{n_m}$ to blockchains for storage. To prevent collusion attacks among users and the single cloud server, the user can introduce more cloud servers, split the key sk_h into more shares, and distribute a share to each cloud server. For example, the key is divided into $(n_c + 1)$ shares $(sk_{h,s,1}, sk_{h,s,2}, \dots, sk_{h,s,n_s}, sk_{h,u})$, each share in $(sk_{h,s,1}, sk_{h,s,2}, \dots, sk_{h,s,n_s})$ is transmitted to a cloud server. Then, during decryption, these cloud servers or above-threshold cloud servers need to collaborate to execute partial decryption.

Check $(u_{id}, sk_{id}, C_m) \rightarrow C_M$: Based on the user identity u_{id} and the ciphertext C , the user checks whether u_{id} satisfies the access policy in C as $u_{id} \in \mathcal{P}$. Then, the user parses the parameter L_{id} as $(O_{id,1}, O_{id,2}, \dots)$, and obtains the parameters in C belonging to him/her (e.g., the index is $j \in [n_p]$), where o denotes the size of L_{id} . Then, the user checks whether an index i exists that satisfies the following equation.

$$e(c_{1,j}, h_{u'_{id,i}}) \stackrel{?}{=} e(O_{id,i}, g) \cdot e(h_{u'_{id,i}}^{sk_{id}}, h_{u'_{id,i}}).$$

If there is no such i , the user executes the **Update** phase for the up-to-date parameter L_{id} . Otherwise, for the index i , the user decrypts the homomorphic secret key pk_h as follows.

$$\begin{aligned}
 \beta &= c_{4,j} \oplus H[(e(O_{id,i}, c_3)^{-1} \cdot c_{2,j})^{\frac{1}{sk_{id}}}], \\
 pk_h || sk_{h,u} &= c_5 \oplus H[e(g, g)^\beta].
 \end{aligned}$$

Next, the user recovers the keys $(pk_h, sk_{h,u})$, encrypts his/her AI model M as C_M , and sends C_M to the cloud server for AI queries. As shown in Algorithm 1 and Algorithm 2, we use decision trees and feed-forward neural networks as two examples to illustrate the model encryption process.

$$C_M = Enc[pk_h, M].$$

Query $(C_M, C_m) \rightarrow ER$: With the encrypted AI model C_M , the cloud server queries the data owners' encrypted dataset $\{c_{7,i}\}_{i=1}^{n_m}$. Specifically, the cloud server executes $Eval[C_M, \{c_{7,i}\}_{i=1}^{n_m}]$ and leverages the parameter $sk_{h,s}$ to gain the encrypted query result ER as

Algorithm 1: Threshold Homomorphic Encryption-based Model Encryption Algorithm (Take Decision Trees as Examples).

Require: Decision tree model $\mathcal{T}$ with internal nodes $\mathcal{N}$ and leaf nodes $\mathcal{L}$. Each internal node $u \in \mathcal{N}$ is associated with a feature index j_u and a threshold θ_u ; each leaf node $\ell \in \mathcal{L}$ is associated with an output value v_ℓ .

Ensure: Encrypted decision tree model $Enc(\mathcal{T})$.

- 1: A decision tree model $\mathcal{T}$ defines a mapping $x \mapsto \mathcal{T}(x)$ by recursively routing the input x from the root to a leaf, where ℓ^* is the reached leaf when following tests $x_{j_u} \stackrel{?}{\leq} \theta_u$ at each node $u \in \mathcal{N}$:

$$\mathcal{T}(x) = v_{\ell^*}.$$

- 2: **for** each internal node $u \in \mathcal{N}$ **do**
- 3: Encrypt the feature index:

$$\tilde{j}_u \leftarrow Enc_{pk}(j_u).$$

- 4: Encrypt the threshold:

$$\tilde{\theta}_u \leftarrow Enc_{pk}(\theta_u).$$

- 5: **end for**
- 6: **for** each leaf node $\ell \in \mathcal{L}$ **do**
- 7: Encrypt the leaf output value:

$$\tilde{v}_\ell \leftarrow Enc_{pk}(v_\ell).$$

- 8: **end for**
- 9: Denote encrypted decision tree parameters as:

$$Enc(\mathcal{T}) = \{\tilde{\theta}_u, \tilde{j}_u\}_{u \in \mathcal{N}} \cup \{\tilde{v}_\ell\}_{\ell \in \mathcal{L}}.$$

- 10: **return** $Enc(\mathcal{T})$.
-

$Dec[sk_{h,s}, Eval[C_M, \{c_{7,i}\}_{i=1}^{n_m}]]$. Next, the result ER is returned to the user. Here, as mentioned in the **Encrypt** algorithm, if the data owner chooses to split the key sk_h to multiple cloud servers, above-threshold cloud servers need to collaborate to execute partial decryption over ER . Finally, the partially decrypted ER is returned to the user. Receiving the encrypted query result ER , the user utilizes the parameter $sk_{h,u}$ to decrypt ER by $Dec[sk_{h,c}, ER]$ and gains the query result R . As shown in Algorithm 3 and Algorithm 4, we use decision trees and feed-forward neural networks as two examples to illustrate the AI query process.

Update (u_{id}) $\rightarrow L_{id}$: Given the user identity u_{id} , the user gains the up-to-date public parameter L_{id} in the public auxiliary parameter aux .

Revoke (u_{id}, pp, aux) $\rightarrow (pp', aux')$: Given the revoked user identity u_{id} , the key curator is responsible for executing user revocation. Specifically, the key curator first generates a revocation factor as $r_{id} \in Z_p$, and then runs the **KeyGen** algorithm to gain the following parameters ($pk_{r,id}, pap_{r,id}$).

$$pk_{r,id} = h_{u'_{id}}^{r_{id}},$$

$$pap_{r,id} = (H_{1,u'_{id}}^{r_{id}}, \dots, H_{u'_{id}-1,u'_{id}}^{r_{id}}, \phi, H_{u'_{id}+1,u'_{id}}^{r_{id}}, \dots, H_{n,u'_{id}}^{r_{id}}).$$

Algorithm 2: Threshold Homomorphic Encryption-based Model Encryption Algorithm (Take Feed-Forward Neural Networks as Examples).

Require: Neural network model $\mathcal{M}$ with L layers and parameters $\{W_\ell, b_\ell\}_{\ell=1}^L$.

Ensure: Encrypted model $Enc(\mathcal{M})$.

- 1: The model $\mathcal{M}$ is a feed-forward neural network with L layers:

$$\mathcal{M}(x) = f_L(W_L \cdot f_{L-1}(\dots f_1(W_1 x + b_1) \dots) + b_L),$$

where f_ℓ are activation functions (e.g., polynomial or piecewise-linear functions that are HE-friendly).

- 2: **for** each layer $\ell = 1$ to L **do**
- 3: **for** each entry $W_\ell(i, j)$ of weight matrix W_ℓ **do**
- 4: Compute encrypted weight:

$$\tilde{W}_\ell(i, j) \leftarrow Enc_{pk}(W_\ell(i, j)).$$

- 5: **end for**
- 6: **for** each entry $b_\ell(k)$ of bias vector b_ℓ **do**
- 7: Compute encrypted bias:

$$\tilde{b}_\ell(k) \leftarrow Enc_{pk}(b_\ell(k)).$$

- 8: **end for**
- 9: **end for**
- 10: Denote encrypted model parameters as:

$$Enc(\mathcal{M}) = \{\tilde{W}_\ell, \tilde{b}_\ell\}_{\ell=1}^L.$$

- 11: **return** $Enc(\mathcal{M})$.
-

Then, towards the user u_{id} , the key curator uses parameters ($pk_{r,id}, pap_{r,id}$) to run the **Register** algorithm and updates the parameters (pp', aux'). Meanwhile, data owners can choose to update the policy-related parameters ($\mathcal{P}', \{c'_{1,i}\}_{i=1}^{n_p}, \{c'_{2,i}\}_{i=1}^{n_p}, c'_3, \{c'_{4,i}\}_{i=1}^{n_p}, c'_5$, stored by the cloud server. After that, ciphertexts on up-to-date parameters ($pp', aux', \mathcal{P}', \{c'_{1,i}\}_{i=1}^{n_p}, \{c'_{2,i}\}_{i=1}^{n_p}, c'_3, \{c'_{4,i}\}_{i=1}^{n_p}, c'_5$ cannot be decrypted by the revoked user, meaning the user's query right is revoked.

C. Detailed Construction for Decart*

Setup (λ) $\rightarrow (crs, pp, aux)$: Given a security parameter λ , the system generates a bilinear group Ψ as (p, g, G, G_T, Z_p, e) , where $e(G, G) \rightarrow G_T$. Let the parameter $N \in Z_p$ as the maximum number of users, $n \in Z_p$ as the number of users in a block, and $B = \lceil \frac{N}{n} \rceil$ as the number of blocks. Under a trusted setup, the system samples a random value $z \in Z_p$, and computes the following parameters.

$$h_i = g^{z^i}, \quad i = 1, 2, \dots, n, n+2, \dots, 2n.$$

Finally, the system selects a hash function $H[G_T] \rightarrow \{0, 1\}^*$ and initializes the common reference string crs , the public parameter pp , and the auxiliary parameter aux as follows.

$$crs = (\Psi, N, B, n, \{h_i\}_{i \in [2n] \setminus n+1}, H[\cdot]),$$

$$pp = (C_{(1)} = 1, C_{(2)} = 1, \dots, C_{(B)} = 1) \in G,$$

$$aux = (L_1 = \{1\}, L_2 = \{1\}, \dots, L_N = \{1\}) \in G.$$

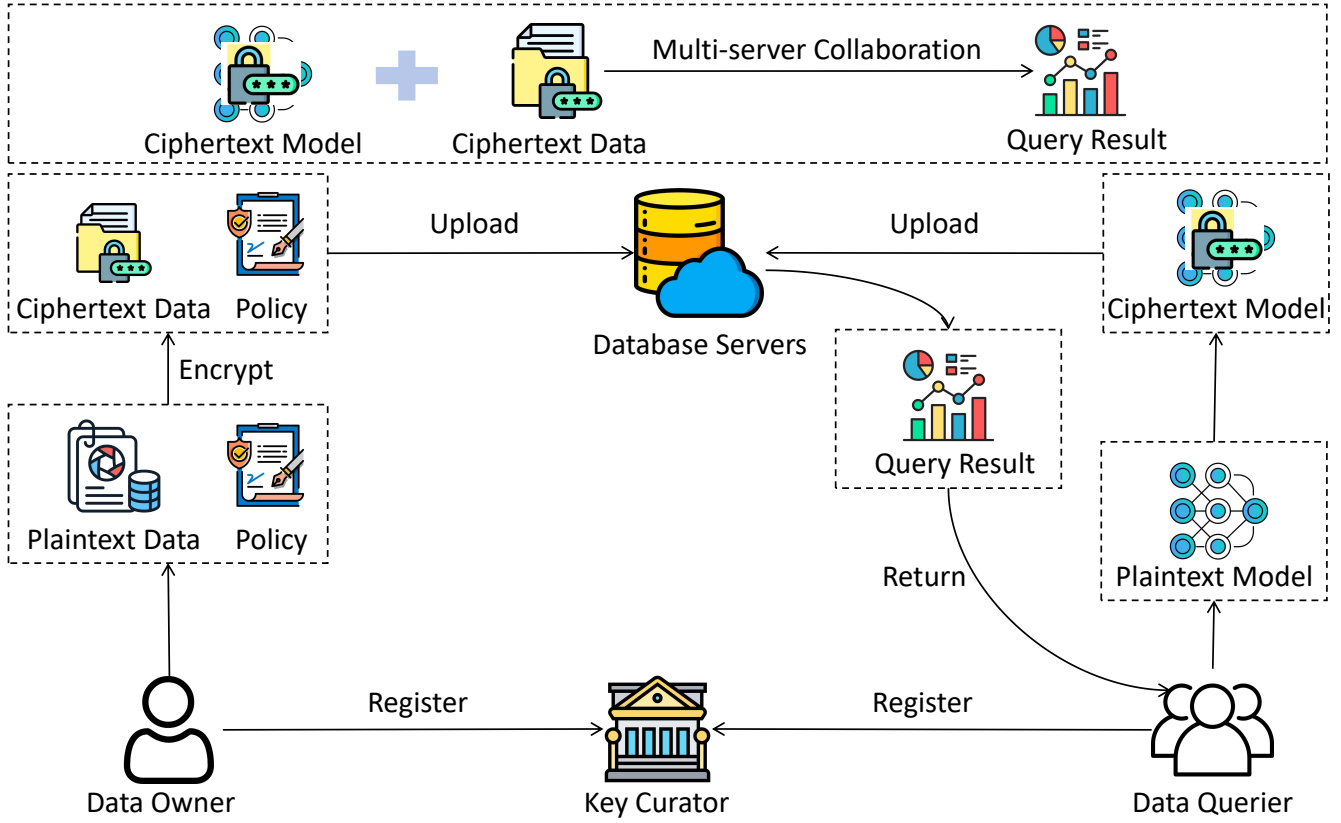


Fig. 3: The workflow of Decart*.

KeyGen (u_{id}) $\rightarrow$ ($sk_{id}, pk_{id}, pap_{id}$): Given the user identity $u_{id} \in Z_p$, the user samples a random value $x_{id} \in Z_p$, and sets $u'_{id} = (u_{id} \bmod n) + 1$. Then, the user generates his/her own secret key sk_{id} , public key pk_{id} , and personal auxiliary parameter pap_{id} as follows. Next, the user sends the tuple $(u_{id}, pk_{id}, pap_{id})$ to the key curator for registration.

$$sk_{id} = x_{id}, \quad pk_{id} = h_{u'_{id}}^{x_{id}},$$

$$pap_{id} = (h_{u'_{id}+n}^{x_{id}}, h_{u'_{id}+n-1}^{x_{id}}, \dots, h_{2+n}^{x_{id}}, \phi, h_n^{x_{id}}, \dots, h_{u'_{id}+1}^{x_{id}}).$$

Register ($u_{id}, pk_{id}, pap_{id}$) $\rightarrow$ (pp', aux'): Receiving tuple $(u_{id}, pk_{id}, pap_{id})$, the key curator checks the user identity u_{id} and personal auxiliary parameter pap_{id} as follows.

$$u_{id} \stackrel{?}{\in} [N],$$

$$e(pk_{id}, h_n) \stackrel{?}{=} e(pap_{id,1}, g) \stackrel{?}{=} \dots \stackrel{?}{=} e(pap_{id,u'_{id}-1}, h_{u'_{id}-2})$$

$$\stackrel{?}{=} e(pap_{id,u'_{id}+1}, h_{u'_{id}}) \stackrel{?}{=} \dots \stackrel{?}{=} e(pap_{id,n}, h_{n-1}),$$

where $u'_{id} = (u_{id} \bmod n) + 1$. If the above checking process holds, the key curator updates the public parameter pp and the auxiliary parameter aux like Decart.**Register** algorithm.

Encrypt ($\mathcal{P}, \{m_i\}_{i=1}^{n_m}$) $\rightarrow C_m$: The user formulates the access policy $\mathcal{P}$ as a series of user identities $\{u_{id,i}\}_{i=1}^{n_p}$, where n_p denotes the size of the access policy. Then, the user samples a random value $\alpha \in Z_p$ and a homomorphic encryption key tuple (pk_h, sk_h) with the homomorphic encryption as $Enc[\cdot]$, homomorphic decryption as $Dec[\cdot]$, and homomorphic evaluation as $Eval[\cdot]$. Then, the user samples two random

value $(\beta, \gamma) \in Z_p$, splits the homomorphic secret key sk_h as two shares $(sk_{h,s}, sk_{h,u})$, and encrypt the data m_i in the dataset $\{m_i \in \{0, 1\}^*\}_{i=1}^{n_m}$ as follows, where n_m denotes the number of data records.

$$k_i = \lceil \frac{u_{id,i} + 1}{n} \rceil, \quad i = 1, 2, \dots, n_p,$$

$$c_{1,i} = C_{(k_i)}, \quad i = 1, 2, \dots, n_p,$$

$$u'_{id,i} = (u_{id,i} \bmod n) + 1, \quad i = 1, 2, \dots, n_p,$$

$$c_{2,i} = e(C_{(k_i)}, h_{n+1-u'_{id}})^\gamma, \quad i = 1, 2, \dots, n_p,$$

$$c_3 = g^\gamma,$$

$$c_{4,i} = H[e(h_{u'_{id}}, h_{n+1-u'_{id}})^\gamma] \oplus \beta, \quad i = 1, 2, \dots, n_p,$$

$$c_5 = H[e(g, g)^\beta] \oplus (pk_h || sk_{h,u}),$$

$$c_{6,i} = Enc[pk_h, m_i], \quad i = 1, 2, \dots, n_m.$$

Finally, the user packs the ciphertext C_m as $(\mathcal{P}, \{c_{1,i}\}_{i=1}^{n_p}, \{c_{2,i}\}_{i=1}^{n_p}, c_3, \{c_{4,i}\}_{i=1}^{n_p}, c_5, \{c_{6,i}\}_{i=1}^{n_m})$, and sends the tuple $(C_m, sk_{h,s})$ to the cloud server. Then, the cloud server retains the policy-related parameters $(\mathcal{P}, \{c_{1,i}\}_{i=1}^{n_p}, \{c_{2,i}\}_{i=1}^{n_p}, c_3, \{c_{4,i}\}_{i=1}^{n_p}, c_5)$ locally and uploads the data-related parameters $\{c_{6,i}\}_{i=1}^{n_m}$ to blockchains for storage. Similar to the **Encrypt** algorithm in Decart, to prevent collusion attacks among users and the cloud server, the user can introduce more cloud servers, split the key sk_h into $(n_s + 1)$ shares $(sk_{h,u}, sk_{h,s,1}, sk_{h,s,2}, \dots, sk_{h,s,n_s})$, and distribute a share $sk_{h,s,i}$ to each cloud server s_i .

Check (u_{id}, sk_{id}, C_m) $\rightarrow C_M$: Based on the user identity u_{id} and the ciphertext C , the user checks whether u_{id} satisfies

Algorithm 3: Encrypted AI Query Algorithm (Take Decision Trees as Examples).

Require: Encrypted data records $\{c_{6,i}\}_{i=1}^{n_m}$, encrypted decision tree model $Enc(\mathcal{T}) = \{\theta_u, j_u\}_{u \in \mathcal{N}} \cup \{\tilde{v}_\ell\}_{\ell \in \mathcal{L}}$, server's key share sk_s , querier's secret share sk_u .

Ensure: Querier obtains the plaintext query results R .

- 1: # Server-side encrypted query:
- 2: **for** each selected encrypted input $c_{6,i}$ **do**
- 3: Let the encrypted feature vector for record i be:

$$\tilde{\mathbf{x}}^{(i)} \leftarrow c_{6,i}.$$

- 4: # Encrypted tree evaluation:
- 5: For each internal node $u \in \mathcal{N}$, the server computes the encrypted comparison result:

$$\tilde{s}_u^{(i)} \leftarrow HE.Eval(g(\cdot), \tilde{x}_{j_u}^{(i)}, \tilde{\theta}_u),$$

where g is an HE-friendly function approximating the indicator of the predicate $[x_{j_u} \leq \theta_u]$ (e.g., a low-degree polynomial).

- 6: Using $\{\tilde{s}_u^{(i)}\}_{u \in \mathcal{N}}$, the server aggregates leaf-node contributions to obtain the encrypted output:

$$\tilde{y}^{(i)} \leftarrow HE.Eval\left(\sum_{\ell \in \mathcal{L}} \alpha_\ell(\{\tilde{s}_u^{(i)}\}_{u \in \mathcal{N}}) \cdot \tilde{v}_\ell\right),$$

where $\alpha_\ell(\cdot)$ is an HE-friendly function that encodes the (soft) routing probability or indicator of reaching leaf ℓ based on the comparison results at internal nodes.

- 7: **end for**
- 8: The server returns the encrypted outputs:

$$ER \leftarrow HE.Dec[sk_s, \{\tilde{y}^{(i)}\}].$$

- 9: **for** each ciphertext $\tilde{y}^{(i)} \in ER$ **do**
- 10: The querier runs the threshold decryption protocol:

$$y^{(i)} \leftarrow HE.Dec(\tilde{y}^{(i)}, sk_u).$$

- 11: **end for**
- 12: **return** $R = \{y^{(i)}\}$.

the access policy in C as $u_{id} \stackrel{?}{\in} \mathcal{P}$. Then, the user parses the parameter L_{id} as $(O_{id,1}, O_{id,2}, \dots)$, and obtains the parameters in C belonging to him/her (e.g., the index is $j \in [n_p]$), where o denotes the size of L_{id} . Then, the user checks whether an index i exists that satisfies the following equation.

$$e(c_{1,j}, h_{n+1-u'_{id}}) \stackrel{?}{=} e(O_{id,i}, g) \cdot e(h_{u'_{id}}^{sk_{id}}, h_{n+1-u'_{id}}).$$

If there is no such i , the user executes the **Update** phase for the up-to-date parameter L_{id} . Otherwise, for the index i , the user decrypts the homomorphic secret key pk_h as follows.

$$\begin{aligned} \beta &= c_{4,j} \oplus H[(e(O_{id,i}, c_3)^{-1} \cdot c_{2,j})^{\frac{1}{sk_{id}}}], \\ pk_h || sk_{h,u} &= c_5 \oplus H[e(g, g)^\beta]. \end{aligned}$$

Next, the user recovers the keys $(pk_h, sk_{h,u})$, encrypts his/her AI mode M as C_M , and sends C_M to the cloud server for AI queries. The model encryption process is similar to the **Check**

Algorithm 4: Encrypted AI Query Algorithm (Take Feed-Forward Neural Networks as Examples).

Require: Encrypted data records $\{c_{6,i}\}_{i=1}^{n_m}$, encrypted model $Enc(\mathcal{M}) = \{\tilde{W}_\ell, \tilde{b}_\ell\}_{\ell=1}^L$, server's key share sk_s , querier's key share sk_u .

Ensure: Querier obtains the plaintext query results R .

- 1: # Server-side encrypted query:
- 2: **for** each selected encrypted input $c_{6,i}$ **do**
- 3: Initialize encrypted activation:

$$\mathbf{h}_0^{(i)} \leftarrow c_{6,i}.$$

- 4: **for** each layer $\ell = 1$ to L **do**
- 5: # Encrypted linear transformation:
For each neuron k in layer ℓ , compute

$$\tilde{z}_{\ell,k}^{(i)} \leftarrow HE.Eval(\tilde{W}_\ell(k, \cdot), \mathbf{h}_{\ell-1}^{(i)}, \tilde{b}_\ell(k)).$$

- 6: Collect ciphertext vector of pre-activations:

$$\tilde{\mathbf{z}}_\ell^{(i)} \leftarrow (\tilde{z}_{\ell,1}^{(i)}, \dots, \tilde{z}_{\ell,m_\ell}^{(i)}).$$

- 7: # Encrypted activation function:
Apply an HE-friendly activation function f_ℓ :

$$\mathbf{h}_\ell^{(i)} \leftarrow HE.Eval(f_\ell, \tilde{\mathbf{z}}_\ell^{(i)}).$$

- 8: **end for**
- 9: At the final layer L , obtain encrypted output:

$$\tilde{y}^{(i)} \leftarrow \mathbf{h}_L^{(i)}.$$

- 10: **end for**
- 11: The server returns the encrypted outputs:

$$ER \leftarrow HE.Dec[sk_s, \{\tilde{y}^{(i)}\}].$$

- 12: **for** each ciphertext $\tilde{y}^{(i)} \in ER$ **do**
- 13: The querier runs the threshold decryption protocol:

$$y^{(i)} \leftarrow HE.Dec(\tilde{y}^{(i)}, sk_u).$$

- 14: **end for**
- 15: **return** $R = \{y^{(i)}\}$.

algorithm in Decart, as shown in Algorithm 1 and Algorithm 2.

$$C_M = Enc[pk_h, M].$$

Update, Query, and Revoke: In Decart*, these algorithms are similar to those of Decart.

D. Correctness Analysis**E. Complexity Analysis****VI. SECURITY ANALYSIS**

In this section, we analyze the security.

VII. PERFORMANCE EVALUATIONS

A. *Settings of Experiments*

Setup.

Datasets.

Baselines for Comparison.

Metric.

B. *Theoretical Complexity*

C. *Experimental Results*

VIII. CONCLUSION

In this paper,

REFERENCES

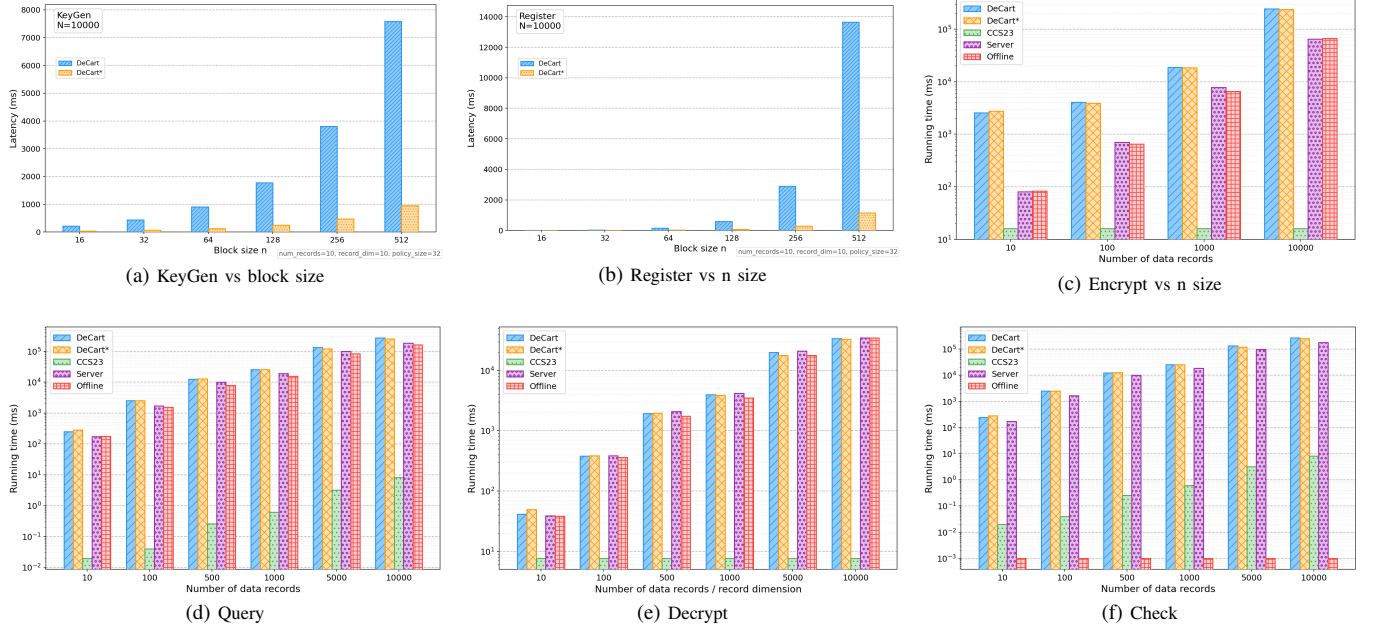


Fig. 4: Computation costs on key generation, register, encrypt, query, decrypt and check.

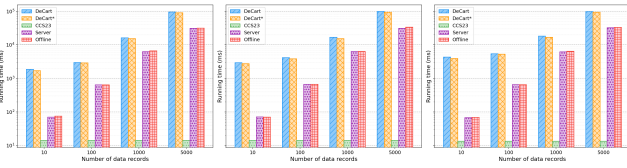


Fig. 5: Encrypt Latency with Different amount of Queriers.

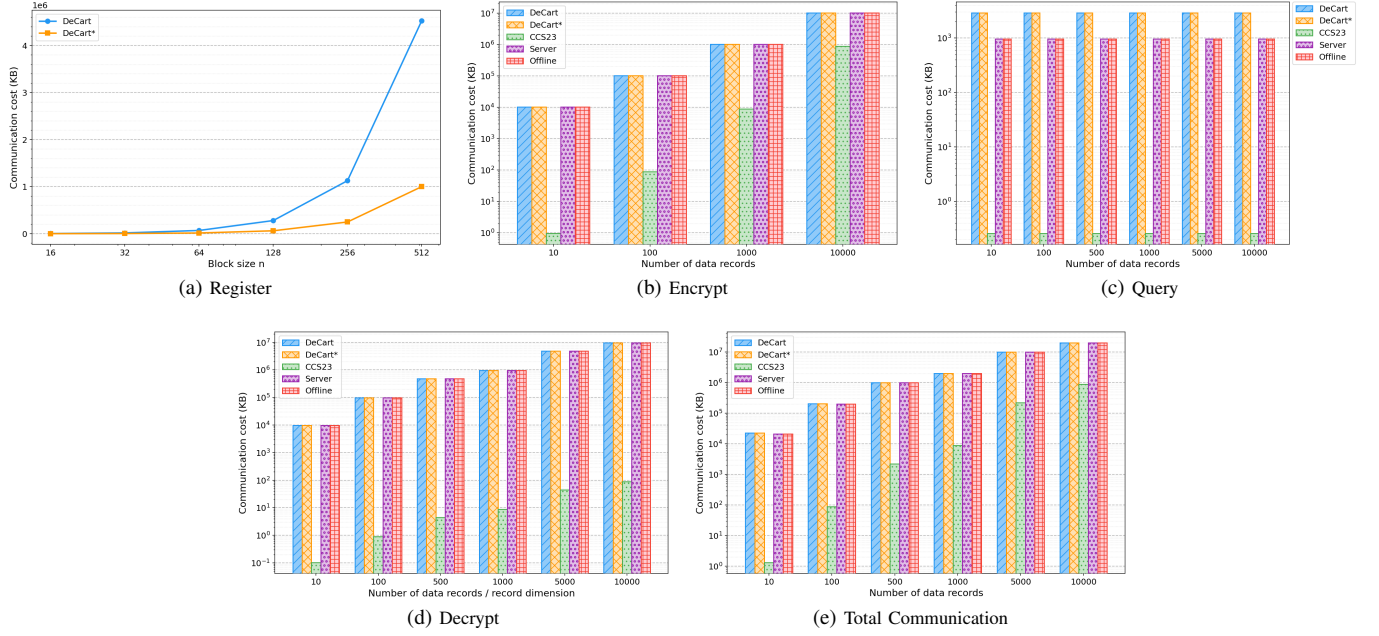


Fig. 6: Communication costs on register, encrypt, query, decrypt and total cost

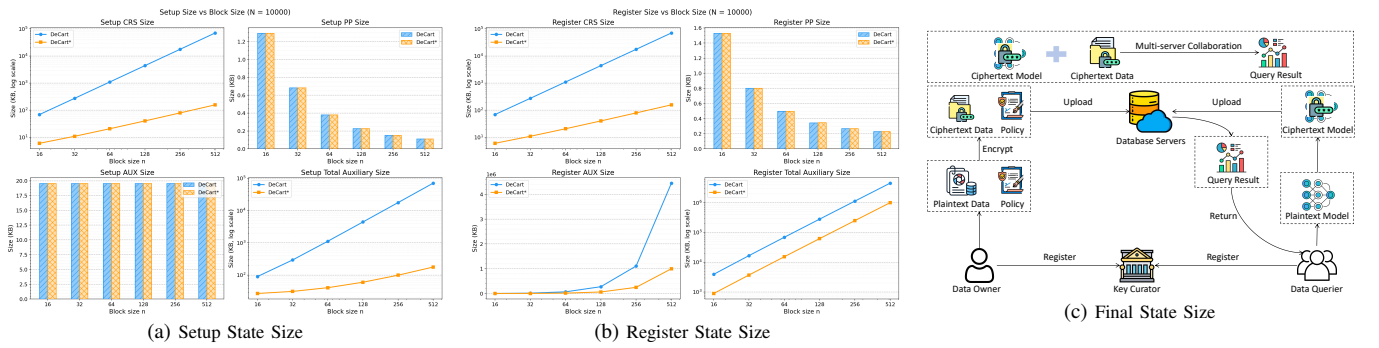


Fig. 7: Structural Size Breakdown Across Different Stages.